



UNIVERSIDAD NACIONAL DE CÓRDOBA

FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES

CÁTEDRA DE ARQUITECTURA DE COMPUTADORAS

TRABAJO PRÁCTICO N° 1

**“Diseño e implementación de una ALU parametrizable en
FPGA”**

Alumnos:

Feiguin Malkoni, Lucía

Monutti, Elena

1. Introducción y Objetivos

El presente informe documenta el desarrollo del Trabajo Práctico N° 1, cuyo objetivo es diseñar e implementar en una FPGA una Unidad Aritmético-Lógica (ALU) parametrizable en su ancho de datos, de forma que pueda ser reutilizada posteriormente en el trabajo final de la materia.

Los objetivos específicos son:

- Implementar el módulo de la ALU en lenguaje Verilog, parametrizado en el ancho del bus de datos (NB_DATA).
- Diseñar una arquitectura de nivel superior (TOP) que separe la lógica combinacional de la secuencial mediante módulos de registro parametrizados (Register).
- Verificar funcionalmente el diseño mediante un testbench con generación de entradas aleatorias y verificación automática de resultados (self-checking testbench).
- Simular el diseño y analizar los tiempos de propagación mediante las herramientas de Vivado.
- Implementar el diseño completo en una placa FPGA, utilizando switches físicos para cargar los operandos A, B y el código de operación, y LEDs para visualizar el resultado.

2. Descripción funcional de la ALU

La ALU implementa ocho operaciones aritméticas y lógicas, seleccionadas mediante un código de operación (OP) de 6 bits. La siguiente tabla resume las operaciones soportadas y su código asociado:

Operación	Código (binario)
ADD	100000
SUB	100010
AND	100100
OR	100101
XOR	100110
SRA	000011
SRL	000010
NOR	100111

2.1 Arquitectura de Hardware en FPGA

Dado que la cantidad de switches de la placa es limitada, se aplica una técnica de multiplexado temporal para ingresar A, B y OP usando un único conjunto de switches:

- Lógica Combinacional (ALU): Bloque libre de clock dedicado exclusivamente al cálculo lógicos/aritmético.
- Lógica Secuencial (Registros de Entrada): Tres instancias de un módulo registro (Register) que capturan el valor de los switches al accionar los botones correspondientes (btn_a, btn_b, btn_op).
- Módulo TOP: Entidad de jerarquía superior dedicada a la interconexión pura entre los registros, la ALU y las E/S físicas de la FPGA.

3. Diseño en Verilog

3.1 Módulo combinacional ALU

La ALU (ALU.v) es un módulo puramente combinacional, libre de reloj (clk), que calcula el resultado RES directamente a partir de las entradas A, B y la operación OP.

- Desplazamiento Aritmético (SRA): Para garantizar que el sintetizador mantenga el bit de signo al desplazar hacia la derecha (preservando el signo en valores negativos), se incluyó el casteo explícito `$signed(A)`.
- Prevención de Latches: Se agregó la sentencia default: `resultado = {NB_DATA{1'b0}}`; dentro del bloque case para cubrir todas las combinaciones posibles del bus OP (6 bits) y evitar la inferencia de elementos de memoria no deseados.
- Manejo de Palabras Reservadas: Las operaciones lógicas se renombraron con el sufijo `_OP` (`AND_OP`, `OR_OP`, `XOR_OP`, `NOR_OP`) dado que las palabras simples son reservadas por el estándar Verilog para compuertas primitivas.
- Implementación de NOR: Se definió mediante la expresión $\sim(A \mid B)$ para aplicar la operación bit a bit sobre dos buses.

3.2 Módulo secuencial Register

Para capturar los valores de los switches físicos, se diseñó un módulo secuencial genérico Register.v parametrizado en su ancho de bus (NB). Posee entrada de reloj clk, reset asíncrono rst y habilitador enable.

En el módulo TOP se instancian tres registros independientes:

- reg_A (8 bits): Habilitado mediante btn_a.
- reg_B (8 bits): Habilitado mediante btn_b.
- reg_OP (6 bits): Habilitado mediante btn_op.

3.3 Módulo superior TOP

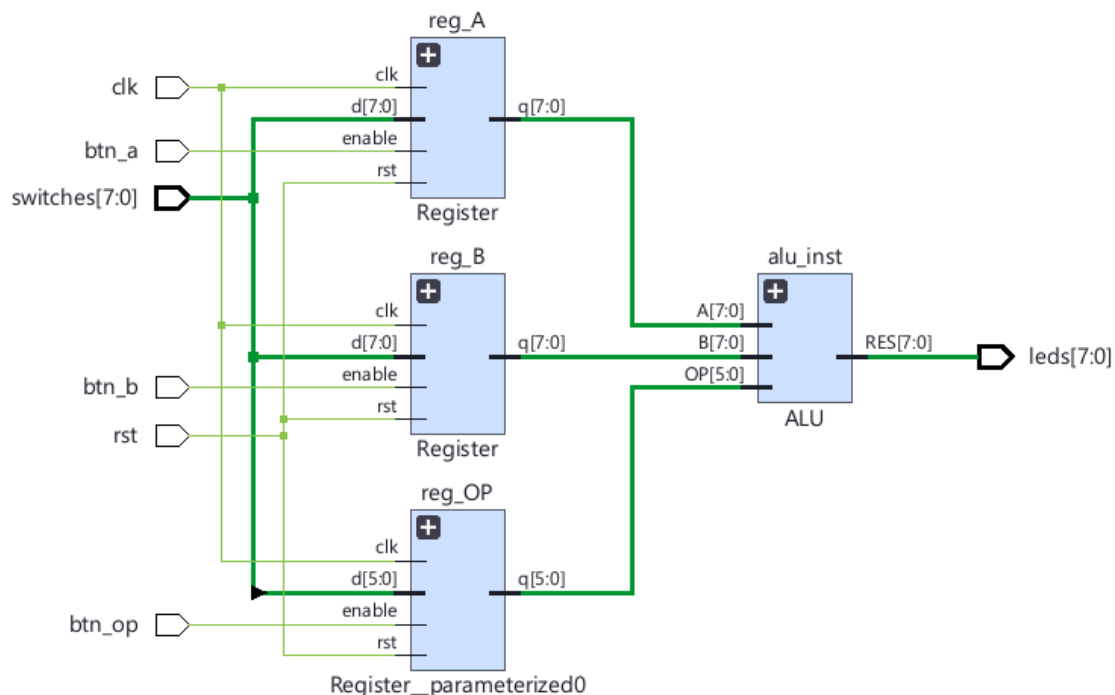
El módulo TOP.v conecta los tres registros con la ALU sin incluir lógica lógica procedimental interna. Al sintetizar e implementar para la FPGA Artix-7 Basys 3 (xc7a35tcbg236-1), Vivado reportó la siguiente utilización de recursos:

Recurso	Utilizado	Disponible	Utilización
Slice LUTs	64	20800	0.31%
Slice Registers	22	41600	0.05%
Bonded IOB	21	106	19.81%
BUFGCTRL	1	32	3.13%

El uso de 22 Flip-Flops responde exactamente al almacenamiento de A (8 bits), B (8 bits) y OP (6 bits). Los 21 pines de E/S corresponden a los 8 switches, 8 LEDs, 4 pulsadores y la señal de reloj.

3.4 Esquemático del sistema

En la Figura 1 se presenta el esquemático RTL generado mediante la herramienta de análisis de Vivado (Elaborated Design) para el módulo superior TOP.



Esquemático RTL del módulo TOP obtenido en Vivado, mostrando la interconexión entre los tres registros de entrada (reg_A, reg_B, reg_OP) y el bloque combinacional alu_inst.

Como se aprecia en el diagrama:

- Multiplexado temporal de entradas: El bus de entrada switches[7:0] se distribuye en paralelo hacia las entradas d de los tres registros de captura.
- Habilitaciones independientes: Las señales btn_a, btn_b y btn_op actúan individualmente sobre los pines enable de cada registro, permitiendo almacenar secuencialmente los datos.
- Acoplamiento directo a la ALU: Las salidas q[7:0] de reg_A y reg_B, junto con q[5:0] de reg_OP, se conectan de manera directa a los puertos de entrada A, B y OP del bloque alu_inst.
- Salida del sistema: El resultado combinacional RES[7:0] se asigna directamente al bus de salida leds[7:0].

4. Verificación funcional (testbench)

4.1 Metodología

1. Prueba Unitaria de la ALU (tb_ALU.v): Verifica el correcto funcionamiento combinacional mediante un modelo de referencia self-checking y generación aleatoria de estímulos.
2. Prueba Unitaria del Registro (tb_Register.v): Confirma la retención de datos con enable = 0, la escritura síncrona con enable = 1 y la respuesta al reset.
3. Prueba de Integración (tb_TOP.v): Emula la interacción completa de un usuario sobre la FPGA accionando secuencias de pulsadores y switches.

4.2 Código del testbench

El testbench de la ALU implementa la función modelo_esperado, la cual calcula el resultado teórico de forma independiente al diseño (DUT). La tarea aplicar_y_chequear aplica 3 pares de operandos aleatorios mediante \$random por cada una de las 8 operaciones (24 pruebas en total).

4.3 Resultados de la simulación

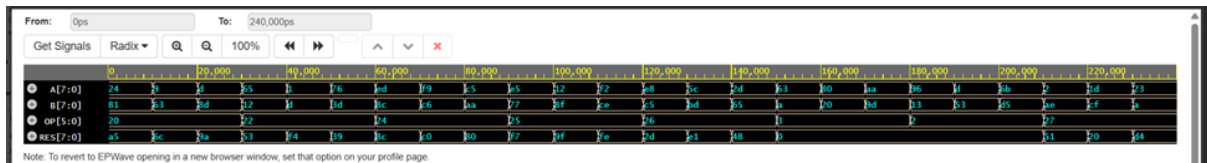
Icarus Verilog / Vivado XSim: Sobre el total de 24 vectores de prueba, no se registraron fallas (0 errores), verificando un 100% de concordancia funcional entre la ALU RTL y el modelo en software.

Observación sobre desplazamientos (SRA / SRL): Al desplazar una cantidad $B \geq 8$ en buses de 8 bits:

- SRL completa con ceros a la izquierda, devolviendo 0.
- SRA conserva el bit de signo mediante \$signed(A). Si el MSB de A es 1, el resultado desplaza unos por la izquierda devolviendo 255 (8'b11111111), validando la preservación aritmética del signo.

4.4 Formas de onda

A continuación se incluye la captura de las formas de onda obtenidas en el visor EPWave, mostrando la evolución temporal de las señales A, B, OP y RES durante la simulación:



Se puede observar cómo, cada 10 ns, cambian los valores de A, B y OP, y cómo el resultado RES se actualiza de forma combinacional (sin retardo de reloj, propio de un diseño puramente combinacional) reflejando la operación correspondiente.

5. Simulación en Vivado y análisis de tiempos

Se realizó la simulación behavioral del sistema completo (tb_TOP) en Vivado 2026.1, utilizando el simulador XSim integrado. El testbench aplicó la secuencia de estímulos correspondiente a cargar A=15 mediante el pulso de btn_a, B=10 mediante btn_b, y el código de operación ADD (6'b100000) mediante btn_op, emulando la interacción esperada con los pulsadores físicos de la placa.

El resultado obtenido en los LEDs fue 25 (15+10), coincidiendo con el valor esperado, por lo que la simulación imprimió el mensaje "INTEGRACION OK: ADD 15 + 10 = 25". La simulación corrió durante 1000 ns sin errores de compilación ni elaboración.

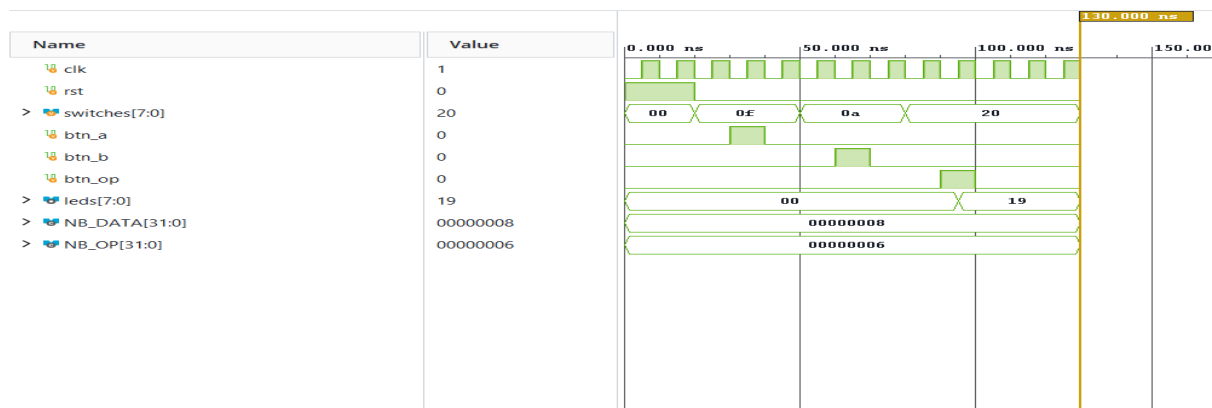


Figura 1: Formas de onda de la simulación behavioral de tb_TOP. Se observa la carga secuencial de switches con A=0x0F (15), B=0x0A (10) y OP=0x20 (código ADD), resultando en leds=0x19 (25), valor esperado de la suma.

Se ejecutó el análisis de tiempos post-implementación con la restricción de reloj de 100 MHz definida en config.xdc. El reporte Design Timing Summary arrojó que la totalidad de las restricciones de temporización especificadas se cumplen ("All user specified timing constraints are met"), sin ningún endpoint fallado (0 de 74 en análisis de setup, 0 de 74 en hold). El slack tanto de setup (WNS) como de hold (WHS) resultó infinito, reflejando el amplio margen disponible dado el bajo nivel lógico combinacional de la ALU frente al período de reloj de 10 ns exigido.

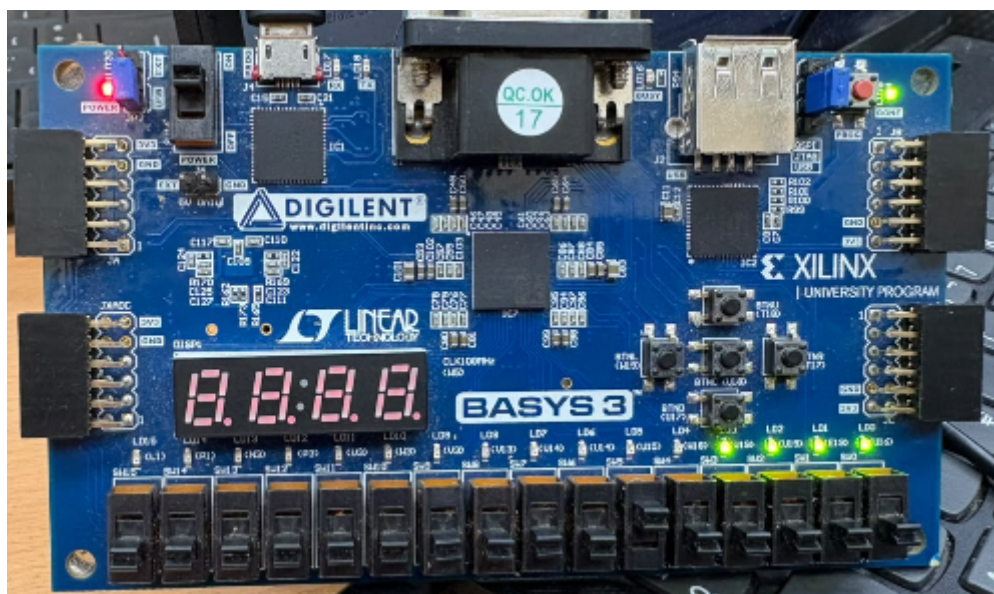
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS)	inf	Worst Hold Slack (WHS)	inf	Worst Pulse Width Slack (WPWS)	4,500 ns
Total Negative Slack (TNS)	0,000 ns	Total Hold Slack (THS)	0,000 ns	Total Pulse Width Negative Slack (TPWS)	0,000 ns
Number of Failing Endpoints	0	Number of Failing Endpoints	0	Number of Failing Endpoints	0
Total Number of Endpoints	74	Total Number of Endpoints	74	Total Number of Endpoints	23

6. Implementación en FPGA

La validación física del sistema completo se llevó a cabo sobre la placa de desarrollo Digilent Basys 3, equipada con el FPGA Xilinx Artix-7 (xc7a35tcpg236-1). A partir del archivo de restricciones físicas (.xdc), se realizó el mapeo de los recursos de E/S de la siguiente manera:

- Entradas de datos (switches[7:0]): Los 8 *switches* deslizantes de la placa se configuraron como el bus compartido para introducir de forma multiplexada los operandos A, B y el código de operación de 6 bits (SW5 a SW0).
- Señales de control y reloj: Los pulsadores físicos se asignaron a las habilitaciones de los registros (BTNU para btn_a, BTND para btn_b, BTNL para btn_op), el botón central (BTNC) se destinó al reset general, y la señal de reloj principal se conectó al oscilador integrado de 100 MHz de la placa (W5).
- Salida de resultados (leds[7:0]): Los 8 LEDs integrados visualizaron de manera directa el bus combinacional RES proveniente de la ALU.

Durante la etapa de pruebas en placa, se operó el sistema mediante la secuencia de carga secuencial: configuración de operandos en los switches, flanco en el pulsador correspondiente para almacenarlos en los registros internos, y posterior selección del código de instrucción.



Detalle de la implementación física en la Basys 3 ejecutando la operación lógica ADD, mostrando el estado de los switches y la correcta representación del resultado en los LEDs indicadores.

Operación realizada:

- A: 00000011
- B: 00001100
- Operación ADD: 100000

7. Conclusiones

El desarrollo del presente trabajo práctico permitió implementar y validar exitosamente una Unidad Aritmético-Lógica (ALU) parametrizable de 8 bits sobre la placa FPGA Digilent Basys 3.

- Coherencia Arquitectónica: La separación modular entre la lógica combinacional (ALU.v) y la secuencial (Register.v) mediante la entidad superior TOP.v demostró ser idónea para entornos con recursos de E/S limitados. La técnica de multiplexado temporal permitió manipular operandos y código de operación mediante un único conjunto de switches físicos sin comprometer la integridad de los datos.
- Verificación Funcional: Las pruebas unitarias e integradas alcanzaron un 100% de efectividad (0 errores en 24 vectores de prueba), confirmando la correcta ejecución de las 8 operaciones aritméticas, lógicas y de desplazamiento, así como el tratamiento del bit de signo mediante \$signed() en desplazamientos aritméticos (SRA).
- Eficiencia de Recursos y Temporización: El diseño sintetizado ocupó apenas el 0.31% de las LUTs y el 0.05% de los registros Slices disponibles en la Artix-7, cumpliendo holgadamente las restricciones de temporización a 100 MHz.
- Validación en Hardware: La prueba física ratificó la concordancia entre los modelos simulados y la síntesis RTL, demostrando que la arquitectura parametrizada queda en condiciones óptimas para ser integrada como subbloque en proyectos de mayor complejidad.